

Claude Certified Architect (Foundations)

Pattern Recognition Cheat Sheet

Exam-day reference. Two sections: domain-specific patterns (organised by the 5 exam domains) and scenario-specific patterns (organised by the 6 recurring exam scenarios). Recognise the pattern; the answer follows.

Universal Reflexes (apply across all questions)

- 1. What's the failure mechanism?** Diagnose before prescribing.
- 2. Is information reaching the layer that needs it?** Check data flow before blaming prompts.
- 3. What are the stakes?** High-stakes → deterministic (hooks, schemas, gates). Low-stakes → prompts acceptable.
- 4. Does this fix work 100% of the time, or just most of the time?** For compliance/financial/security, only 100% is acceptable.
- 5. Am I treating a categorical problem with a quantitative tweak?** If yes, wrong fix.
- 6. Is the answer applying the right intervention at the right layer?** Upstream fixes for upstream bugs; preprocessing for input issues; runtime enforcement for guarantees; prompts for interpretation.
- 7. 'What worked correctly' sentences = evidence of innocence.** The failure is in the unmentioned component.
- 8. Cross-scenario recognition is faster than option elimination.** Identify the scenario in the first 1-2 sentences, then match the symptom signature.

Part 1: Domain-Specific Patterns

Five domains, organised by exam weighting. Each row maps a question signature to its answer family.

Domain 1: Agentic Architecture & Orchestration (27%)

IF THE QUESTION DESCRIBES...	THE ANSWER INVOLVES...
Agent loop terminating prematurely; checking content[0].type == 'text'	Replace with stop_reason == 'tool_use' / 'end_turn' check
Agent loop using iteration cap as primary stop signal	Iteration cap is safety net only; stop_reason is primary
Agent parsing natural language ('I'm done') to detect completion	Use stop_reason field, never NL parsing
Multi-agent report missing entire topic categories; subagents worked correctly	Coordinator's task decomposition is the root cause
Subagent doesn't have access to coordinator's earlier findings	Pass info explicitly in subagent's prompt; no shared memory exists
Coordinator can't spawn subagents at all	allowedTools missing 'Task '
5 subagents running sequentially; latency too high	Emit multiple Task calls in single coordinator response → parallel
Need to explore two divergent approaches from shared analysis	fork_session
High-stakes rule (refund/transfer/compliance) violated despite prompt instruction	Tool-call interception hook on the action tool
Mandatory first step (extract metadata, verify identity) sometimes skipped	tool_choice: {'type': 'tool', 'name': 'X'} for forced selection
Heterogeneous tool data confusing the model	PostToolUse hook to normalise formats
Handoff to human agent — human reading conversation transcript	Self-contained handoff package: customer ID, summary, root cause, recommendation
Multi-issue customer request handled one at a time; slow	Decompose into parallel investigation, synthesise unified resolution
Predictable structured task with same steps every time	Fixed sequential pipeline (prompt chaining)
Open-ended investigation; right path depends on findings	Dynamic adaptive decomposition
Code review 14+ files: inconsistent depth, missed bugs, same pattern flagged/skipped	Multi-pass architecture : per-file local + separate cross-file integration pass
Parallel per-file passes proposed; no integration pass	Wrong answer — missing integration pass cannot catch cross-file issues
Resume session after small change (~<25% files changed)	Resume + targeted re-read of changed files
Resume session after large change (~>50% files changed)	Fresh start with summary injection
Session resumed; agent gives advice based on stale tool results	Inform agent of specific file changes for targeted re-analysis

Domain 2: Tool Design & MCP Integration (18%)

IF THE QUESTION DESCRIBES...	THE ANSWER INVOLVES...
Two tools with similar/vague descriptions; model misroutes	Expand descriptions with purpose, inputs, examples, edge cases, explicit boundaries
Single tool does many different things (extract, summarise, verify); poor accuracy	Split into purpose-specific tools with defined contracts
Misrouting; routing classifier proposed as fix	Wrong — over-engineering; better descriptions first
Misrouting; few-shot examples proposed as fix	Wrong — works around bug; description quality fixes root cause
Tool returns empty array for both 'no matches' and 'couldn't reach DB'	Structurally distinguish: <code>{success: true, data: []}</code> vs <code>{isError: true, errorCategory: 'transient'}</code>
Tool returns <code>isError: true</code> for a valid empty result (no matches exist)	Tool misclassifying valid empty as error; fix at tool layer
Refund exceeds policy limit; tool refuses	<code>errorCategory: 'business'</code> , <code>isRetryable: false</code> , customer-friendly description, suggested action
Network timeout calling third-party API	<code>errorCategory: 'transient'</code> , <code>isRetryable: true</code>
Invalid input format passed to tool	<code>errorCategory: 'validation'</code> , <code>isRetryable: true</code> (after fix)
Access denied error	<code>errorCategory: 'permission'</code> , <code>isRetryable: false</code> , escalate
Agent has 15-20 tools; selection degraded	Reduce to 4-5 tools scoped to role
Synthesis subagent has web search tools	Wrong scope — remove out-of-role tools
Synthesis frequently round-trips to coordinator for verification (85% simple)	Scoped cross-role tool : give synthesis a constrained <code>verify_fact</code> tool
Need guaranteed structured output; model picks from multiple schemas	<code>tool_choice: 'any'</code>
Must call specific tool first (e.g., <code>extract_metadata</code>)	<code>tool_choice: {'type': 'tool', 'name': 'extract_metadata'}</code>
Team wants Jira/GitHub/Slack integration; dev proposes custom MCP server	Evaluate community MCP servers first ; build custom only for unmet needs
Custom build justified with 'we want full control' / 'fine-grained control'	Wrong — vague preferences don't justify custom build
MCP credentials in <code>.mcp.json</code> shared across team	Use <code>\${VAR_NAME}</code> environment variable expansion
Agent prefers built-in Grep over more capable MCP code-search server	Enhance MCP tool description with explicit 'use this instead of Grep when...'
Finding files containing specific function calls	Grep (searches contents)
Finding files matching naming pattern (<code>*.test.tsx</code>)	Glob (matches paths)
Edit failing because text isn't unique	Fall back to Read + Write full file
Agent reading every file upfront to understand codebase	Wrong — context-budget killer; use Grep for entry points, then Read selectively

Domain 3: Claude Code Configuration & Workflows (20%)

IF THE QUESTION DESCRIBES...	THE ANSWER INVOLVES...
CLAUDE.md scope: applies to YOU only (not shared)	User-level: <code>~/ .claude/CLAUDE.md</code>
CLAUDE.md scope: shared with team via git	Project-level: <code>.claude/CLAUDE.md</code> or root <code>CLAUDE.md</code>
New team member missing team standards (others get them correctly)	Standards in user-level config; move to project-level CLAUDE.md
Project CLAUDE.md uses <code>@import</code> ; new dev's Claude doesn't load conventions	Imported files not committed; commit <code>.claude/rules/</code> files
Developer's personal preference imposed on whole team via project CLAUDE.md	Move to user-level <code>~/ .claude/CLAUDE.md</code>
Need to debug which memory files actually loaded	/memory command
200-line task-specific procedure bloating CLAUDE.md; slowing every interaction	Move to skill/command (on-demand) — task-specific doesn't belong in always-loaded
Always-applied standard placed in skill that needs invocation	Wrong mechanism — move to CLAUDE.md (automatic)
<code>/review</code> command available to whole team	<code>.claude/commands/review.md</code> (project-scoped, version-controlled)
Personal <code>/brainstorm</code> shouldn't impose on teammates	<code>~/ .claude/commands/</code> or <code>~/ .claude/skills/</code>
Skill produces verbose intermediate output polluting main conversation	context: <code>fork</code> frontmatter
Skill should be analytical only; no writes	<code>allowed-tools: ['Read', 'Grep', 'Glob']</code>
Skill needs to prompt for missing required parameter	<code>argument-hint: 'What component?'</code>
Test conventions for test files spread across 50+ directories	Path-specific rule with paths: <code>['**/*.test.{ts,tsx}']</code>
Standards for files in <code>terraform/</code> and subdirectories	Path-specific rule with paths: <code>['terraform/**/*']</code> — directory CLAUDE.md doesn't recurse
Migration conventions; migration files in single directory only	Still prefer path-specific rule with glob (recursion, type filtering, future-proof)
Restructure monolith → microservices, 200 files, undecided boundaries	Plan mode
Fix single null pointer at known line	Direct execution
Rename function across 38 call sites in 19 files (consistency matters)	Hybrid: plan mode for survey/strategy, direct execution for application
Library migration affecting 30 files with consistent pattern	Hybrid: plan for strategy, direct for application
Prose description interpreted inconsistently across runs	Few-shot examples with concrete input/output pairs
Developer in unfamiliar domain; doesn't know edge cases	Interview pattern — Claude asks clarifying questions
Implementation task with well-specified testable behaviour	Test-driven iteration
Multiple fixes that affect each other (changing A affects B)	Single batched message
Multiple independent fixes in different files	Sequential iteration
CI pipeline runs <code>claude 'Analyze this PR'</code> ; hangs indefinitely	Add -p flag (print mode — non-interactive single-shot)
Need machine-parseable CI output for posting as PR comments	<code>--output-format json</code> with <code>--json-schema</code>
Same Claude session generates code and reviews it; quality poor	Two independent Claude sessions — fresh context for review
CI re-runs review; keeps flagging same issues already addressed	Pass prior review findings in context; instruct to report only new/unaddressed issues
CI-generated tests are low-value boilerplate	Document testing standards in project CLAUDE.md so CI-invoked Claude reads them

Domain 4: Prompt Engineering & Structured Output (20%)

IF THE QUESTION DESCRIBES...	THE ANSWER INVOLVES...
Vague instruction 'be conservative' / 'high-confidence findings'	Replace with categorical criteria (specific include/exclude rules)
One category has 35-40% false positive rate; developers losing trust	Temporarily disable the bad category; iterate against labelled set; restore trust
Confidence threshold proposed as fix for inconsistent output	Wrong — confidence-based instructions are vague antipattern
Severity levels defined by prose ('Critical: serious bugs')	Replace with concrete code examples for each severity
Detailed prompt; inconsistent formatting across runs	Few-shot examples (2-4 targeted at ambiguous cases)
Inconsistent classification on borderline multi-category cases	Few-shot examples specifically targeting the borderline cases with reasoning
Extraction misses fields present in source; hallucinates citations	Few-shot examples showing varied document structures with correct extractions
Many examples covering every possible case proposed	Wrong — 2-4 targeted examples beat 20 exhaustive ones
Claude occasionally produces malformed JSON	tool_use with JSON schema (eliminates syntax errors)
tool_use configured; schema syntax correct; but values fabricated	tool_use doesn't prevent semantic errors; fix elsewhere (schema design)
Required schema field fabricated when source lacks the data	Change field to optional/nullable in schema
Schema enum forces model to pick closest type for documents that don't fit	Add 'other' enum value + freeform detail string
Documents genuinely ambiguous between defined enum types	Add 'unclear' enum value with criteria
Validation-retry fixes format errors (date wrong format, missing field)	Effective use of validation-retry
Validation-retry on fabricated values when source lacks the data	Wrong tool — retry can't fix genuine information absence; fix schema (nullable)
Two failure patterns: format mismatch AND genuine absence	Different fixes for each: retry for format, schema change for absence
Same retry produces different fabricated values each time	Diagnostic for genuine information absence
Need 50% cost savings; workload is overnight reports / weekly audits	Batch API (latency-tolerant)
Need 50% cost savings; workload is PR review CI / real-time chat	Wrong — keep sync; batch's 24-hr SLA breaks blocking workflows
Manager proposes batch for all workloads to save cost	Selective: batch latency-tolerant only, sync for blocking
Batch volume-based prioritisation (highest volume → batch)	Wrong — volume doesn't override latency constraints
Same session generates and reviews code; misses issues	Two independent Claude sessions (architectural separation)
Review session uses larger context to hold generation reasoning	Wrong — amplifies the bias
Model self-reports confidence per finding; routed to humans for low-confidence	Calibrate against labelled validation set ; legitimate confidence use
Confidence as vague prompt instruction ('only flag confident findings')	Antipattern — vague threshold

Domain 5: Context Management & Reliability (15%)

IF THE QUESTION DESCRIBES...	THE ANSWER INVOLVES...
Long conversation; exact values lost (refund \$247.83, medication doses, contract clauses)	Persistent case facts block included verbatim every prompt, never summarised
Reduce summarisation frequency proposed as fix	Wrong — quantitative tweak to categorical problem
Larger context window proposed to avoid summarisation	Wrong — capacity fix to structural problem; lost-in-middle still applies
Prompt instruction 'preserve exact values' proposed	Wrong — can't recover information already destroyed by summarisation
Long document (50-100 pages); middle sections missed	Lost in the middle — restructure: lead with key findings, explicit section headers
Tool returns 40+ fields when only 5 needed	Trim tool results at execution layer before appending to context
Subsequent API requests omit earlier messages to save tokens	Wrong — full conversation history required every call
Downstream agent context overloaded by upstream verbose output	Upstream agent optimisation — return structured facts/citations, not prose
Customer says 'I want a human' on first turn	Escalate immediately ; no retention; no investigation; complete handoff package
Customer frustrated about simple issue; no human request	Acknowledge frustration + offer resolution
Customer reiterates human request after agent offered help	Escalate — explicit reiterated request is the trigger
Escalation based on sentiment intensity proposed	Wrong — frustration doesn't correlate with case complexity
Escalation based on self-reported confidence proposed	Wrong — model confidently wrong on hard cases
Multiple customer records match 'John Smith'; agent picks most recently active	Wrong — heuristic disambiguation; ask for additional identifiers
Subagent returns 'success' with partial results; omits failed sources	Structured error propagation : failure type, what attempted, partial results, alternatives
Single component failure terminates entire pipeline	Workflow termination antipattern — propagate with partial results
Synthesis output looks complete but actually missing data from failed source	Coverage annotations — surface gaps transparently
Long Claude Code session; references 'typical patterns' instead of specific classes	Context degradation — use scratchpad files (externalise findings)
Agent crash mid-session; work in progress lost	Manifest file with structured state; load into fresh session on resume
96% aggregate accuracy; team wants to fully automate	Segment metrics by document type, vendor, field; stratified sampling
High-confidence extractions never reviewed	Sample high-confidence cases too — catches novel error patterns
Synthesis loses source attribution between agents	Structured claim-source mappings preserved through synthesis
Two credible sources report different statistics	Annotate both with attribution and dates; let consumer decide
Synthesis averages conflicting statistics	Wrong — methodologies differ; averaging produces meaningless number
Synthesis picks more recent or 'more authoritative' source automatically	Wrong — surface the conflict transparently
Numbers from different time periods reported as contradictions	Temporal awareness — require publication/data collection dates

Universal Wrong-Answer Patterns (auto-eliminate)

When you see these phrasings in answer options, eliminate them unless the scenario specifically justifies them.

WRONG-ANSWER PHRASE	WHY IT IS WRONG
'Use a more powerful model'	Capability doesn't fix architectural problems
'Increase context window'	Capacity doesn't fix structural problems; lost-in-middle still applies
'Add a confidence threshold to the prompt'	Vague instruction antipattern
'Be more conservative / careful / critical'	Vague instruction in disguise
'Strengthen prompt with explicit instructions' (high-stakes rule)	Probabilistic can't enforce non-negotiable rules
'Train developers to filter unreliable findings'	Trust isn't filterable once eroded
'Reduce timeout / retry count / summarisation frequency'	Quantitative tweak to categorical problem
'Switch from tool_use to prompt-based JSON'	Abandons working syntax mechanism
'Pick ambiguous match by recency/activity heuristic'	Heuristic disambiguation — ask for identifier
'Sentiment-based escalation'	Frustration ≠ case complexity
'Self-reported confidence routing' (as primary signal)	Confidently wrong on hard cases
'Reinstall Claude Code'	Runtime fix for configuration problem
'Hold a training session'	Configuration problem, not knowledge gap
'Terminate workflow on any failure'	Throws away partial results; orchestration should decide
'Have downstream detect what upstream silently suppressed'	Wrong layer; fix at the failing component
'Open-source the custom build'	Doesn't change whether the build was justified
'Switch the highest-volume workload to batch'	Volume ignores latency constraints
'Bypass MCP, use fetch tool directly'	Regresses to fragile prompt-heavy pattern
'Average conflicting statistics'	Methodologies differ; averaging is meaningless

Part 2: Scenario-Specific Patterns

The exam clusters questions around 6 recurring scenarios. Recognise the scenario in the first 1-2 sentences, then match the symptom signature to find the answer family quickly.

Scenario 1: Customer Support Resolution Agent

IF THE QUESTION DESCRIBES...	THE ANSWER INVOLVES...
Refund processed without verifying account ownership (8% rate)	Tool-call interception hook on process_refund requiring verify_account_ownership first
Refund over \$500 occasionally approved without manager review	Tool-call interception hook with amount check; blocks and escalates if > \$500
Sanctions screening skipped on 0.3% of transfers	Tool-call interception hook on process_international_transfer requiring sanctions check
Strengthened system prompt proposed for high-stakes verification	Wrong — prompt-layer probabilistic; need runtime hook for 100% enforcement
Few-shot examples proposed for high-stakes compliance rule	Wrong — still prompt-layer; can't reach zero failure rate
Customer says 'I want a human' on first turn	Escalate immediately — no investigation, no retention, complete handoff package
Customer frustrated about simple issue; no explicit human request	Acknowledge frustration + offer resolution
Customer reiterates human request after agent offered help	Escalate — reiterated request is the trigger
Sentiment-based escalation proposed ('frustration intensity')	Wrong — frustration ≠ case complexity
Self-reported confidence routing ('if confident, continue')	Wrong — model often confidently wrong on hard cases
Multiple 'John Smith' records; agent picks most recently active; refund applied to wrong customer	Ask customer for additional identifier (email, phone, order ID)
Heuristic disambiguation (recency, similarity, activity) on ambiguous matches	Wrong — produces confidently-wrong outcomes
Long conversation; exact values lost (\$247.83, order #8891, March 3rd)	Persistent case facts block included verbatim every prompt; never summarised
Progressive summarisation destroys numerical/date/ID precision	Case facts block; conversation history can still summarise for narrative
Tool returns empty array for both 'no matches' and 'DB unreachable'; agent retries unnecessarily	Tool layer must distinguish: success with empty data vs isError with errorCategory
Tool returns isError: true for customer with no subscriptions (valid empty result)	Fix at tool layer; return success with empty data, not transient error
Multi-concern request handled sequentially (slow)	Decompose into parallel investigation; synthesise unified resolution
Handoff to human agent — human reading conversation transcript	Self-contained handoff package : customer ID, summary, root cause, refund amount, recommendation

SCENARIO REFLEX: 'Customer support agent' + 'high-stakes action' (refund, transfer, account closure) → **hook-based enforcement**.
'Long conversation losing details' → **case facts block**. 'Multiple matches' → **ask for additional identifier**.

Scenario 2: Code Generation with Claude Code

IF THE QUESTION DESCRIBES...	THE ANSWER INVOLVES...
Generated code doesn't follow team naming conventions	Conventions in project CLAUDE.md (shared via git)
New team member's code generic; established team gets conventions correctly	Conventions in user-level; move to project-level CLAUDE.md
@import references in CLAUDE.md don't resolve for new team members	Imported files not committed; commit <code>.claude/rules/</code> files
Developer's personal 'show alternatives' preference imposed on whole team	Move from project CLAUDE.md to user-level <code>~/ .claude/CLAUDE.md</code>
200-line API endpoint procedure bloating CLAUDE.md; slowing every interaction	Move to skill/command (on-demand); CLAUDE.md is for always-applied standards
<code>/generate-component</code> available to whole team	<code>.claude/commands/</code> (project-scoped)
Personal <code>/scaffold</code> skill with verbose exploration output	<code>~/ .claude/skills/</code> (user-scoped) with context: fork
Test files spread across 50+ directories; consistent generation conventions needed	Path-specific rule: paths: <code>['**/*.test.{ts,tsx,js,jsx}']</code>
Migration files across services with consistent conventions	Path-specific rule with paths: <code>['**/migrations/**/*.sql']</code>
Standard 'all Python files must have type hints'	Project CLAUDE.md (always-applied); not a skill (which requires invocation)
Refactor monolith → microservices, 200 files, undecided boundaries	Plan mode
Fix typo in error message string at known line	Direct execution
Rename <code>parseDate</code> → <code>parseDateString</code> across 38 call sites in 19 files	Hybrid: plan for survey/strategy, direct for application
Library migration affecting 30 files with consistent pattern	Hybrid mode
Prose description of code transformation produces inconsistent results	Few-shot examples with concrete input/output
Multiple independent fixes in different files	Sequential iteration
Multiple fixes that affect each other	Single batched message
Developer in unfamiliar domain (distributed systems, consensus)	Interview pattern — Claude asks clarifying questions
Generated code consistently misses edge cases	Few-shot examples covering edge cases; interview pattern if requirements unclear

SCENARIO REFLEX: 'Claude Code generating' + 'inconsistency across developers' → **CLAUDE.md scope problem**. 'Transformation produces different results each run' → **few-shot examples**. 'Task description with file counts' → **plan/direct/hybrid decision**.

Scenario 3: Multi-Agent Research System

IF THE QUESTION DESCRIBES...	THE ANSWER INVOLVES...
Report missing entire topic categories; subagents worked correctly on inputs they received	Coordinator's task decomposition is the root cause
Report missing geothermal/tidal when query was 'renewable energy'	Coordinator decomposition didn't generate subtopics for missing categories
Search subagent's queries too narrow proposed as fix when subagents worked correctly	Wrong — search did its job on queries it was given; bug is upstream
Synthesis subagent dropped content proposed when synthesis worked correctly	Wrong — can't synthesise what wasn't passed to it
Synthesis produces report without source attribution	Structured claim-source mappings preserved by coordinator; not flattened to prose
Coordinator flattens [{claim, url, title}] into bullet list before passing to synthesis	Preserve structure: pass structured data to synthesis with attribution instruction
Synthesis frequently round-trips to coordinator for fact verification (85% simple)	Scoped cross-role tool : give synthesis a constrained verify_fact tool
Give synthesis full web search toolset proposed	Wrong — tool overload and role bleed; use scoped version
Coordinator can't spawn subagents at all despite delegation prompt	allowedTools missing 'Task'
5 subagents running sequentially; latency too high	Emit multiple Task calls in single coordinator response → parallel
Subagent doesn't have access to coordinator's earlier findings	Pass info explicitly in subagent prompt; subagents have no shared memory
Synthesis subagent has web search tools	Wrong scope — remove out-of-role tools
Web search returns verbose content + reasoning; synthesis context overloaded	Upstream agent optimisation : return structured facts/citations, not prose
Document analysis fails silently on corrupted PDF; returns 'success' with partial results	Structured error propagation : failure type, what attempted, partial results, alternatives
Single subagent failure terminates entire pipeline	Workflow termination antipattern — propagate with partial results
Synthesis output looks complete but missing data from failed source	Coverage annotations — surface gaps transparently
Two credible sources report different statistics (14% vs 17%)	Annotate both with attribution and dates; don't pick arbitrarily
Synthesis averages conflicting statistics (15.5%)	Wrong — methodologies differ
Synthesis picks more recent or 'more authoritative' source automatically	Wrong — surface conflict transparently; let consumer decide

SCENARIO REFLEX: 'Multi-agent research' + 'missing content categories' → **coordinator decomposition**. 'Lost source attribution' → **coordinator flattening structured data**. 'Synthesis quality issues' → **trace upstream** (was it given correct inputs?). 'Subagent failed but pipeline produced output' → **silent suppression antipattern**.

Scenario 4: Developer Productivity with Claude

IF THE QUESTION DESCRIBES...	THE ANSWER INVOLVES...
Need to find all callers of a function	Grep for the function name (content search)
Need to find all test files matching pattern (*.test.tsx)	Glob with naming pattern (path search)
Find files importing a module; then find their test files	Grep first (find importers); Glob second (find related test files)
Glob with **/*functionName* proposed to find function callers	Wrong — files aren't named after the functions they call
Agent reading every file upfront to understand codebase	Wrong — context-budget killer; Grep for entry points, then Read selectively
Edit failing because text isn't unique	Fall back to Read full file + Write modified version
Wrapper modules re-exporting functions; single Grep misses callers	Grep export chain first (find re-export points); then Grep each variant
45-turn session; Claude references 'typical patterns' instead of specific classes found earlier	Context degradation — scratchpad files externalise findings
Larger context window proposed for context degradation	Wrong — capacity fix; doesn't address structural cause
Prompt instruction 'reference specific classes' proposed for context degradation	Wrong — can't recover degraded information through instructions
Restart session entirely without preserving findings	Wrong — throws away work; use fresh start with summary injection if needed
Broad codebase question; subagents miss scheduled jobs / CLI / admin actions	Revise coordinator planning to identify plausible entry points before synthesis
Subagents told to 'read every file matching X terms' as fix for missing coverage	Wrong — exhaustive read antipattern; coordinator decomposition is the bug
Fixed pipeline ('always run controller + database + CLI + worker subagents')	Wrong — different queries need different entry points; needs dynamic decomposition
Long exploration session loses grip on earlier findings	Scratchpad files + subagent delegation for noisy work + /compact
Agent crash mid-session; work lost	Manifest export to known location; load into fresh session on resume
Agent prefers Grep over more capable MCP code-search server	Enhance MCP tool description: 'use this instead of Grep when...'

SCENARIO REFLEX: 'Developer exploring codebase' + 'Grep vs Glob choice' → **content vs name pattern question**. 'Long Claude Code session' + 'lost specifics' → **context degradation**, scratchpad files. 'Broad codebase question' + 'missed categories' → **coordinator decomposition**.

Scenario 5: Claude Code for CI/CD

IF THE QUESTION DESCRIBES...	THE ANSWER INVOLVES...
CI pipeline runs <code>claude 'Analyze this PR'</code> ; hangs indefinitely	Add -p flag (print mode — non-interactive single-shot)
Pipe newline into Claude proposed as fix	Wrong — works around mode mismatch; doesn't fix it
Increase CI job timeout proposed for hanging pipeline	Wrong — process is in wait state; not slow processing
Reduce prompt scope proposed for hanging pipeline	Wrong — same misdiagnosis
Need machine-parseable CI output for posting as PR comments	<code>--output-format json</code> with <code>--json-schema</code>
CI-generated tests are low-value boilerplate; don't use fixtures	Document testing standards in project CLAUDE.md so CI-invoked Claude reads them
Same Claude session generates code and reviews it in CI; review quality poor	Two independent Claude sessions — fresh context for review
Self-review with larger context window proposed	Wrong — amplifies confirmation bias
Strengthen review prompt with 'be critical' proposed	Wrong — prompt can't override structural bias
CI re-runs review on updated PR; keeps flagging same issues	Pass prior review findings in context; instruct to report only new/unaddressed issues
Code style category 38% false positive rate; developers ignoring all findings	Temporarily disable the bad category; iterate against labelled set; restore trust
Add confidence threshold to all categories proposed	Wrong — vague instruction antipattern
Reduce verbosity of findings proposed	Wrong — quality problem, not overwhelm problem
Train developers to filter unreliable findings proposed	Wrong — trust isn't filterable once eroded
PR review CI blocking developer merges; manager proposes batch API for cost savings	Keep synchronous — batch's 24-hr SLA breaks blocking workflows
Overnight test generation; weekly compliance audit	Batch API (latency-tolerant)
Real-time customer support chat	Synchronous (blocking)
Manager proposes batch for all workloads	Selective: latency-tolerant only; sync for blocking
Switch highest-volume workload to batch for biggest savings	Wrong — volume doesn't override latency constraints
Severity calibration: prose definitions of 'critical / high / medium'	Concrete code examples for each severity
Code review explicit criteria: 'be conservative' / 'high-confidence findings'	Replace with categorical: 'flag bugs and security; skip style preferences'

SCENARIO REFLEX: 'CI' + 'hanging' → **-p flag missing**. 'CI generating low-value output' → **CLAUDE.md missing context**. 'Same session generated and reviewed' → **split into two sessions**. 'Blocking workflow' + 'batch for cost savings' → **keep synchronous** regardless of cost.

Scenario 6: Structured Data Extraction

IF THE QUESTION DESCRIBES...	THE ANSWER INVOLVES...
Malformed JSON output occasionally breaks parsers	tool_use with JSON schema (eliminates syntax errors)
tool_use configured; JSON syntax always correct; but values fabricated	tool_use doesn't prevent semantic errors; fix elsewhere
Required schema field fabricated when source lacks data (PO number, tax ID, regulatory ID)	Change field to optional/nullable in schema
Switch from tool_use to prompt-based JSON proposed	Wrong — abandons working syntax mechanism; doesn't fix fabrication
Prompt instruction 'do not fabricate, write NOT FOUND' proposed	Wrong — prompt vs schema conflict; nullable field is the structural fix
Confidence threshold 'only extract confident fields' proposed	Wrong — vague instruction + worsens under-extraction
Schema enum forces model to pick closest type for documents that don't fit	Add 'other' enum value + freeform detail string
Documents genuinely ambiguous between defined enum types	Add 'unclear' enum value with criteria
Date field accepts both 'Jan 15' and '2024-01-15'	Schema for structure + prompt for format normalisation
Validation-retry resolves format errors 92% of the time	Effective use — information is in source; retry surfaces it correctly
Validation-retry on fabricated values produces different fabricated values each retry	Wrong tool — genuine information absence; fix schema (nullable)
Two failure patterns: format mismatch + genuine absence	Different fixes for each: retry for format, nullable schema for absence
Increase retry count from 1 to 4 for fabrication problem	Wrong — quantitative fix to categorical problem
Improve error messages for genuine absence cases	Wrong — Claude has no info to act on regardless of message quality
Extraction misses fields in varied document structures; hallucinates citations	Few-shot examples showing varied document structures with correct extractions
Prose description of extraction rules producing inconsistent results	Few-shot examples (2-4 targeted at ambiguous cases) with reasoning
Add stricter validation rules in prompt proposed for hallucination	Wrong — negative instruction can't fix capability gap
Switch to more capable model proposed	Wrong — capability doesn't fix configuration gap (missing examples)
Force tool selection on first call (extract_metadata before enrichment)	tool_choice: {'type': 'tool', 'name': 'extract_metadata'}
Internally inconsistent extracted values (line items don't sum to total)	Self-correction flow: extract both stated_total and calculated_total; flag discrepancies
Need to track which code construct triggered a finding	detected_pattern field in structured output
96% aggregate accuracy; team wants full automation without human review	Segment metrics by document type, vendor, field; stratified sampling on production
High-confidence extractions never sampled for review	Sample across confidence levels — catches novel error patterns
Confidence threshold chosen arbitrarily for routing to human review	Calibrate against labelled validation set
Need to process large volume of documents; no real-time requirement	Batch API (50% cost savings, up to 24-hr SLA)
Failed batch documents — resubmit all or just failures?	Resubmit only failures (identified by custom_id) with modifications
Need to refine extraction prompt before processing 10,000 documents	Refine prompt on small sample set BEFORE batch processing

SCENARIO REFLEX: 'Extraction' + 'fabricated fields' → check required fields; **make them nullable**. 'Extraction' + 'varied document formats' → **few-shot examples**. 'Validation-retry not working' → check if genuine information absence (retry can't fix) vs format mismatch (retry works). 'tool_use configured; fabrication happening' → semantic errors aren't prevented by tool_use; **fix at schema layer**.

Cross-Scenario Recognition Reflex

Identify the scenario quickly from the first 1-2 sentences of the question:

QUESTION OPENING	SCENARIO
'Customer support agent...'	Scenario 1
'Claude Code generating...' / 'code review system...'	Scenario 2 or 5
'Coordinator and subagents...'	Scenario 3
'Developer exploring codebase...'	Scenario 4
'CI pipeline...' / 'CI job...'	Scenario 5
'Extraction system with schema...'	Scenario 6

BIGGEST EXAM ADVANTAGE: Scenario recognition + symptom signature is faster than option-by-option elimination. Use the options to **verify** your answer, not to **find** it. Once you've identified the scenario, the family of likely failure modes narrows dramatically. Then look for the specific symptom signature in this cheat sheet — you'll often have the answer family before you finish reading the options.